Para llevar al labo

Lo mínimo para entender y hacer cada tarea de la próxima reunión de laboratorio

Armado por Matías Bélico · 26 de octubre de 2026 · Al día con el contexto maestro §7.1 (versión 2.3)

Una ficha por tarea, en el orden de la reunión. Cada una dice qué es, qué vamos a hacer, qué tiene que dar y con qué tener cuidado.

Esto no reemplaza al repo: si algo de acá no coincide con el repo, manda el repo. Cada ficha cierra con un recuadro **"Para seguir leyendo en el repo"**. Las imágenes de internet llevan un recuadro naranja **"En nuestra placa es distinto"** donde no coinciden con lo nuestro: no copies el dibujo, copiá la corrección. Los tres códigos nuevos (C1, C2a y C2b) están marcados **SIN PROBAR**.

Seguridad: antes de tocar nada

1. **Nada se conecta a un riel sin medirlo antes.** Ningún motor se mueve antes de V1, la seta y los fusibles (fichas 3, 4 y 5).
2. **Fuente de banco siempre limitada en corriente.** Arrancá con el límite bajo y subilo de a poco.
3. **La placa no se protege sola de la polaridad invertida**: Q1 está puenteado. Alimentala solo por el XT60, nunca con pinzas ni cables sueltos.
4. **No enchufes el USB del ESP32 montado en la placa** hasta hacer la corrección de la ficha 2. La única excepción es la medición de 5 s de esa ficha.
5. **ADS1115 a 3,3 V, y ninguna entrada por encima de 3,3 V.**
6. **Protoboard solo en la prueba en banco** (ESP32 suelto con el ACS712 y el BTS7960). Por el protoboard van señales, nunca la corriente del motor. En el rover: conectores con traba, nada de Dupont.
7. **Cortes y soldaduras, sin batería.** La masa de la punta del osciloscopio va siempre a GND: está unida a la tierra de la red.
8. **Un motor trabado pide ~5,5 A.** Un sentido por vez y las manos lejos de la rueda.

> **SOBRE LOS V1 A V12**
> Cuando una ficha nombra V1, V3, V7…, se trata de **hipótesis de escritorio que hay que confirmar o descartar midiendo, no fallas**. Lo que se mida se anota en `pendientes-de-verificacion.md`.

El orden de la reunión

Ficha	Tema	Tarea §7.1	Qué cierra o desbloquea
1	Los dos LM2596 (qué es un buck)	#1	Que no lleguen 12 V al ESP32 ni a los servos
2	Retroalimentación del USB y el Schottky	#2	Programar sin riesgo
3	V1: el driver 4 durante el arranque	#3	V1
4	Seta y relé	#4	V7
5	Fusibles	#5	Condición para energizar la tracción
6	Rieles con el osciloscopio	#6	V3; validación de los rieles
7	BTS7960 (puente H) y código C1	#10 y #7	Reparar o comprar el módulo que falló
8	ACS712 y códigos C2a y C2b	#7	Cadena de medición
9	Bus I2C y ADS1115	#7	Cadena de medición
10	TMCS1126: código de lectura	#8	V9, V11, calibración
11	Encoders con PCNT (guía de lectura)	#11	Encoders validados

Del 6 en adelante se puede trabajar en paralelo. La tarea #9 (cables) no tiene ficha. Antes de ir: revisá que el IDE tenga el **core Arduino-ESP32 3.x** (C1 y los ejemplos del PCNT no compilan con la 2.0.x; en la notebook donde se armó esta guía hay una 2.0.17).

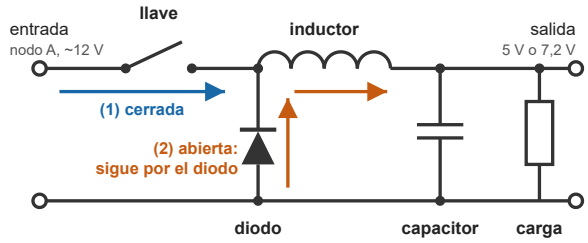
1 Los dos LM2596: qué es un buck

Un buck baja la tensión prendiendo y apagando una llave muy rápido. Si la llave se rompe en corto, la salida copia la entrada: 12 V directo al ESP32.

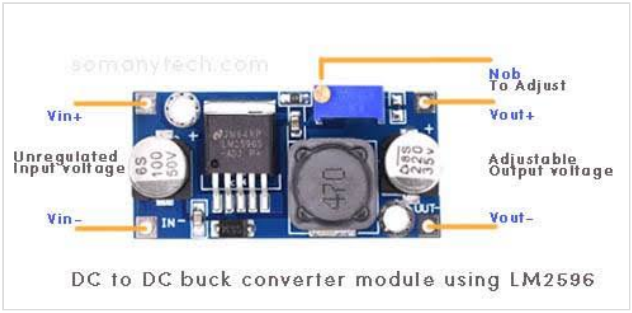
QUÉ ES

Un convertidor reductor (buck) tiene cuatro piezas: una **llave**, un **inductor**, un **diodo** y un **capacitor**. Con la llave cerrada, la entrada empuja corriente por el inductor hacia la salida (1). Con la llave abierta, el inductor no deja que esa corriente se corte de golpe y la sigue empujando, ahora por el diodo (2). El capacitor alisa. La salida depende de qué fracción del tiempo está cerrada la llave. Como casi no disipa, pierde poco: un regulador lineal, en cambio, quema la diferencia en calor.

Por qué hay dos: U2 en **5 V**, solo para el ESP32 (y, por el AMS1117 del devkit, todo el 3,3 V). El segundo, en **7,2 V** y solo para los servos (8-oct): así un servo trabado hunde su propio riel y no reinicia el ESP32 (V3).



Buck simplificado. (1) Llave cerrada: la entrada alimenta la salida y carga el inductor. (2) Llave abierta: el inductor sigue empujando corriente por el diodo.



El módulo que usamos: bornes de entrada (IN+, IN−), de salida (OUT+, OUT−) y el preset azul para ajustar. IN− y OUT− son la misma masa. Imagen: somanytech.com. Sirve tal cual.

QUÉ VAMOS A HACER

1. Fuente de banco en 12 V, **limitada a ~300 mA**, con la salida del módulo desconectada.
2. **Sin carga:** el consumo tiene que ser de pocos mA y la salida, la ajustada.
3. **Con carga**: ~22 Ω en el de 5 V (son ~0,23 A y ~1,1 W: resistencia de 2 W o más). Mirá la regulación y el rizado con el osciloscopio.
4. Repetir con el otro módulo. Si hay que ajustar: el de servos queda en 7,2 V. U2 queda en 5 V hasta poner el Schottky de la ficha 2; ahí pasa a ~5,4 V.

TIENE QUE DAR

Sin carga, pocos mA y la salida en el valor ajustado. Con carga, la salida casi no se mueve.

MALA SEÑAL

La salida copia la entrada: la llave está en corto, el módulo está muerto y no se conecta. También: la fuente entra en limitación sin carga, o el módulo calienta sin carga. Hay que comprar un repuesto igual.

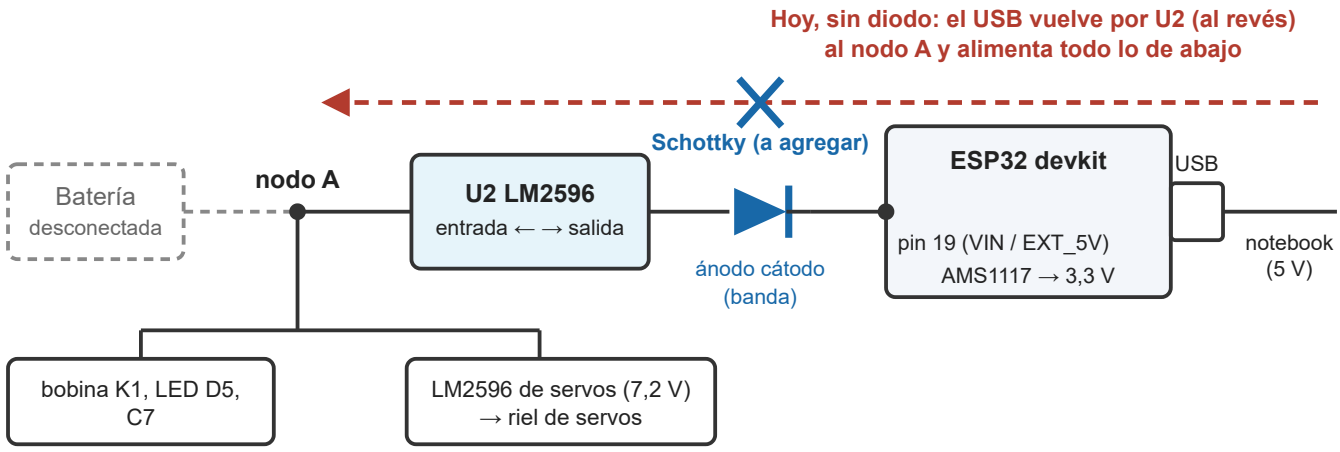
CUIDADO CON

Los dos módulos calentaron el 23-oct con el USB (ficha 2): pueden estar dañados aunque parezcan andar. El preset es multivuelta: hay que girarlo varias vueltas para notar el cambio, y se gira con la salida medida.

Para seguir leyendo en el repo `20-modelo-escala/20-electronica/arquitectura-electrica.md` (cadena de potencia, rieles y presupuesto del 3,3 V) · `direccion-swerve-servos.md` (por qué el segundo LM2596, V3)

2 Retroalimentación del USB y el Schottky

Con la batería desconectada, el 5 V del USB entra al revés por U2 y alimenta media placa. Un diodo Schottky entre U2 y el ESP32 corta ese camino.



El camino del USB. El pin 19 está unido directo a la salida de U2. Un LM2596 no está hecho para recibir tensión en la salida con la entrada sin alimentar, y conduce hacia su entrada. El Schottky, con el ánodo del lado de U2, deja pasar de U2 al ESP32 y bloquea la vuelta.

QUÉ ES

Con el nodo A en ~4 V, se alimentan la bobina del relé, el LED, C7 y el LM2596 de servos. Este último queda con la llave siempre cerrada, porque su salida está ajustada en 7,2 V y nunca la alcanza. Así calentaron los módulos el 23-oct.

Por qué un Schottky: cae poco, 0,3 a 0,4 V a la corriente del ESP32. Aun así hay que **subir U2 a ~5,4 V** para que VIN quede en ~5,0 V: si no, el AMS1117 queda con 1,3 V de margen, justo en su caída mínima.

QUÉ VAMOS A HACER

1. **Confirmar el camino:** batería desconectada, USB enchufado, medir el nodo A **durante 5 s como máximo**. Si hay varios voltios, es este camino.
2. **¿El devkit trae diodo en VBUS?** Todo desenergizado, medir en modo diodo entre el VBUS del conector USB y el pin 5V del devkit.
3. **Poner el diodo** (corte y puente): 1N5819 (con patas, más fácil para un puente) o SS34 (SMA). **Llevarlo.**
4. Subir U2 a ~5,4 V (con la ficha 1).

TIENE QUE DAR (LA VERIFICACIÓN FINAL)

USB enchufado y batería desconectada: **nodo A en 0 V y módulos fríos.** Con batería: VIN en ~5,0 V.

MALA SEÑAL

Sigue habiendo tensión en el nodo A: el diodo está al revés o hay otro camino. VIN por debajo de ~4,6 V: al AMS1117 le falta margen (necesita 1,1 a 1,3 V sobre 3,3 V).

CUIDADO CON

- La **banda del diodo es el cátodo** y va hacia el ESP32.
- Si el devkit **no** tiene diodo en VBUS, con batería y USB enchufados a la vez U2 puede empujar corriente hacia la notebook. No enchufes las dos cosas juntas hasta saberlo.
- Esta tarea es un **corte y puente** sobre una placa ya soldada: cortá sin batería y revisá el corte con el multímetro antes de soldar.

Para seguir leyendo en el repo `20-modelo-escala/20-electronica/arquitectura-electrica.md`, recuadro "Problema encontrado el 23-oct-2026" · `00-estado-actual.md` (reunión del 23-oct)

3 V1: el driver 4 durante el arranque

Tarea #3 · Cierra: V1

IO5 es un pin de arranque que empieza en alto, y por U1 maneja R_PWM_4. Hay que ver con el osciloscopio que el motor 4 quede frenado y no girando mientras el ESP32 arranca.

QUÉ ES

Pines de arranque (strapping): el ESP32 lee IO0, IO2, IO5, IO12 e IO15 en los primeros milisegundos para decidir cómo arranca, así que no se les puede colgar cualquier carga. IO5 arranca en alto y, a través de U1 (NLAS4157), pone R_PWM_4 en alto. Para que el motor no gire, el 27-sep se soldó un **pull-up en L_PWM_4**: con las dos entradas en alto, el motor queda frenado (es el estado "BURN" de la foto de la ficha 7, que no quema nada). **Falta verificarlo.**

Disparo en un flanco: el osciloscopio congela la pantalla cuando una señal cruza un nivel en un sentido. Si se dispara en la subida de EN (el reset del ESP32), lo que queda a la derecha del disparo es todo el arranque.

QUÉ VAMOS A HACER

1. Driver 4 **sin potencia** (cable de alimentación del driver 4 desconectado).
2. Canal 1 en EN del devkit: disparo por flanco de subida, en modo de disparo único. Canal 2 en R_PWM_4 (pin 2 de J19); repetir con L_PWM_4 (pin 1). Con 4 canales, las tres a la vez.
3. Base de tiempo para ver alrededor de 1 s. Apretar el botón EN.
4. Después, con el motor conectado y la fuente limitada: encender y confirmar que no se mueve.

TIENE QUE DAR
R_PWM_4 y L_PWM_4 en alto, claramente por encima del umbral, desde que aparece el 3,3 V hasta que el firmware toma las líneas.

MALA SEÑAL
L_PWM_4 en bajo o a media altura con R_PWM_4 en alto: el motor 4 giraría solo.

CUIDADO CON
El umbral que importa es el de la entrada del módulo. Si el módulo trae el buffer 74HC244 (ficha 7), con VCC a 3,3 V pide unos 2,3 V (0,7 de su alimentación, interpolando la hoja de datos). Referite siempre al GPIO por su nombre, no por el número de pin del devkit: los números cambian según el dibujo.

Para seguir leyendo en el repo `20-modelo-escala/40-firmware/mapa-gpio.md` (pines de arranque y NLAS4157; es la fuente única del pinout) · `20-electronica/drivers-bts7960.md` (estado del canal 4) · `30-pcb/mapa-conectores.md` (J19)

4 Seta y relé

Tarea #4 · Cierra: V7

La seta corta la bobina del relé, el relé abre y muere el bus de los drivers. Solo la tracción: servos y ESP32 siguen vivos, y es a propósito.

QUÉ ES

Un **relé** es una llave movida por un electroimán. Mientras circula corriente por la **bobina** (~150 mA en K1), el contacto **NA** (normal abierto) se cierra y el **NC** (normal cerrado) se abre. La seta es un pulsador NC en serie con la bobina: al apretarla se corta la bobina, se abre el NA y se corta el riel de 12 V conmutado. Si se corta un cable de la seta, también se para: falla segura. D3, en antiparalelo con la bobina, absorbe el pico de tensión al cortarla.

QUÉ VAMOS A HACER

1. **Sin alimentación:** continuidad entre el nodo A y el riel conmutado. Tiene que dar **abierto** (V7).
2. Alimentado: medir el riel conmutado (en C2 o en la entrada de un driver). Apretar la seta: clic y el riel a 0 V. Soltarla: vuelve.

TIENE QUE DAR
Abierto en el paso 1; el riel cae a 0 al apretar y vuelve al soltar. El LED (en el nodo A) sigue prendido.

MALA SEÑAL
Continuidad en el paso 1, o el riel no cae: contactos soldados por el arco (V7). La seta dejaría de cortar sin que nadie se entere.

CUIDADO CON
Sin drivers conectados, C2 (1000 µF) se descarga solo por el divisor de batería (10 k + 2,2 k): son ~12 s. No es una falla: medí con un driver conectado o esperá. Cuál par de K1 es la bobina se dedujo del símbolo: conviene confirmarlo con el multímetro.

Para seguir leyendo en el repo `20-electronica/protecciones-desacople.md` (capas de protección, V6 y V7) · `arquitectura-electrica.md` ("La seta corta solamente la tracción")

5 Fusibles

Tarea #5 · Desbloquea: energizar la tracción

El fusible protege el cable, no la carga. Primero se mide el cable; recién después se elige el fusible.

QUÉ ES

El fusible se elige para que corte antes de que el **cable** se recaliente. Por eso depende de la sección del conductor, que se da en **AWG**: número más chico, cable más grueso. Un fusible **retardado (T)** aguanta picos cortos, como el arranque de un motor (que por un instante pide su corriente de trabado), y corta si la sobrecorriente sigue.

QUÉ VAMOS A HACER

1. Pelar una punta y medir con **calibre el conductor** del cable de ~1,5 mm (placa a drivers) y el de batería.
2. Elegir con la tabla. Previsto: **F3..F6 de 7,5 A retardados** (5×20 mm), uno por driver. **General junto al XT60:** 20 A retardado con 14 AWG o más grueso; 15 A con 16 AWG.

Qué era "1,5 mm"	Sección	Equivale	¿7,5 A lo protege?
Diámetro del conductor	~1,8 mm²	~15 AWG	Sí, con margen
Sección de 1,5 mm²	1,5 mm²	~16 AWG	Sí, con margen
Diámetro con aislación	~0,3 mm²	~22 AWG	**No**

MALA SEÑAL

Si era el diámetro con aislación (~22 AWG), 7,5 A no lo protege, y bajar a 5 A cortaría en cada trabado de una rueda (5,5 A). Hay que rehacer esos tramos con cable más grueso.

CUIDADO CON

Fusible y driver **no son correlativos**: F5 → driver 1, F3 → driver 2, F4 → driver 3, F6 → driver 4. Si el cable no entra en el agujero, **no cortes hilos**: baja la sección que el fusible supone.

Para seguir leyendo en el repo `20-modelo-escala/30-pcb/mapa-conectores.md` ("Cable soldado en J1 y J4..J7: calibre y fusible") · `30-pcb/bom.md` (valores de fusible)

6 Rieles con el osciloscopio

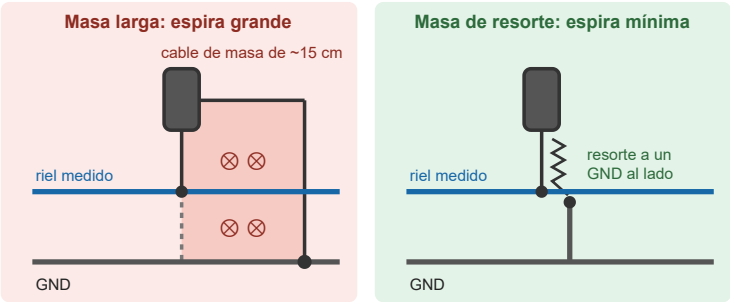
Tarea #6 · Cierra: V3 y la validación de los rieles

Con motores y servos andando, cada riel tiene que aguantar firme. Medí con la masa de resorte: el cable de masa largo agrega ruido que en el riel no existe.

QUÉ ES

- **Rizado:** la ondulación que deja la conmutación, en régimen.
- **Sobrepico:** un salto por encima del valor en un transitorio, por ejemplo al frenar o invertir un motor (su energía vuelve al bus).
- **Caída:** el riel se hunde cuando arranca algo que pide mucha corriente (un motor o un servo, que al arrancar piden casi su corriente de trabado).

Por qué la masa de resorte: la punta y su cable de masa forman una espira. Los motores y la conmutación generan campo magnético, que induce tensión en esa espira, y en la pantalla aparece como si estuviera en el riel. El resorte achica la espira casi a cero.



Izquierda: el área sombreada capta el campo de los motores (⊗) y suma ruido falso. Derecha: el resorte toca un GND a milímetros del punto medido.

Riel	Dónde medir	Qué mirar	Qué esperar (repo)
Bus de 12 V conmutado	En C2 o en la bornera de un driver	Sobrepicos y caídas al arrancar e invertir	Batería de plomo: 10,5 a 14,4 V
Riel de servos	Detrás de F2, en C1	Caída con los 4 servos moviéndose	7,2 V menos la caída del PTC
5 V (VIN)	Pin VIN (5V) del devkit	Que no caiga cuando arrancan motores o servos	5 V hoy; ~5,0 V con el Schottky
3,3 V	Pin 3V3 del devkit	Que no caiga nunca: si cae, se reinicia el ESP32 (V3)	3,3 V

CÓMO DISPARAR Y QUÉ TIENE QUE DAR
Disparo en el mismo riel: flanco de bajada, unos cientos de mV debajo del valor nominal, en modo único; después se arranca el motor. Acople en continua para caídas; en alterna para ver rizado. Tiene que dar: 3,3 V y 5 V firmes con todo andando.

MALA SEÑAL
El 3,3 V se hunde o el ESP32 se reinicia cuando se mueven los servos (V3). El 5 V queda debajo de ~4,6 V en un arranque: el AMS1117 se queda sin margen.

CUIDADO CON
La masa de la punta es la tierra de la red. Si la conectás a un punto que no es GND y la fuente o la notebook también están a tierra, hacés un corto. Siempre a GND.

Para seguir leyendo en el repo `20-electronica/arquitectura-electrica.md` (tabla de rieles) · `direccion-swerve-servos.md` (V3 y el ensayo de servos trabados) · `30-pcb/pendientes-de-verificacion.md`

7 BTS7960: el puente H

Tareas #10 y #7 · Desbloquea: reparar o comprar el módulo que falló

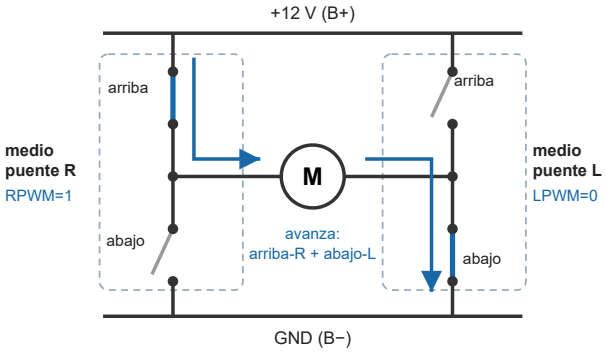
Un puente H son cuatro llaves con el motor en el medio. Según qué par se cierra, el motor avanza, retrocede o frena. Con EN fijo en 3,3 V, como en la placa, no hay rueda libre.

QUÉ ES

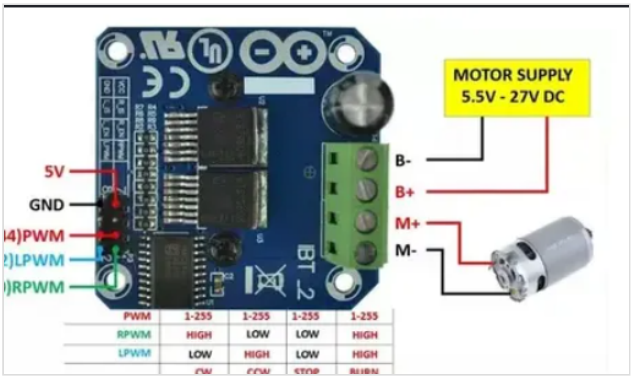
Cerrando la llave de arriba a la izquierda y la de abajo a la derecha, la corriente cruza el motor en un sentido; con las otras dos, en el otro. Cada **chip BTS7960 es medio puente** (una llave arriba y una abajo) y el módulo trae dos: uno lo maneja **RPWM** y el otro **LPWM**. Cada entrada elige qué llave de su medio puente se cierra: **alta, la de arriba; baja, la de abajo**. El chip mismo impide que se cierren las dos de un lado a la vez. Con PWM en una entrada y la otra en bajo, el motor ve tensión una fracción del tiempo: eso da la velocidad.

RPWM	LPWM	Qué pasa (con EN en alto)
1	0	Avanza (la imagen dice CW)
0	1	Retrocede (CCW)
0	0	Frena contra masa: las dos de abajo cerradas (STOP)
1	1	Frena contra la batería: las dos de arriba cerradas. **La imagen dice "BURN": no quema nada**
EN = 0		Rueda libre: todo abierto. **No existe en nuestra placa**

Por qué en la placa los EN están fijos a 3,3 V: ahorran 4 GPIO. A cambio, el reposo depende solo de las líneas PWM (de ahí V1), y la única forma de apagar los puentes por hardware es la seta.



Las cuatro llaves. Cada recuadro punteado es un chip BTS7960 (medio puente). En azul, el camino de la corriente para "avanza".



Módulo BTS7960 (IBT-2) con su tabla de verdad. Imagen de internet, fuente no identificada.

EN NUESTRA PLACA ES DISTINTO

1. **"BURN" no quema.** RPWM y LPWM en alto cierran las dos llaves de arriba: el motor queda frenado. Es justo el estado seguro que usa la placa en el arranque (pull-up de L_PWM_4, V1).
2. **La imagen pone VCC a 5 V y EN con PWM.** En la placa, VCC lógico y EN van a **3,3 V fijos** y la velocidad va por RPWM y LPWM. En el protoboard, igual que en la placa.

QUÉ VAMOS A HACER (CÓDIGO C1, EN EL ANEXO)

1. Armado en banco: ESP32 suelto, **la placa queda libre**. Conexión en el encabezado de C1. Fuente limitada en corriente.
2. **Primero C1 con un módulo que anda:** así el armado queda validado (es "el canal que ya anduvo" de §7.1).
3. Después, **el módulo que falló**, sin cambiar nada más. Un sentido por vez: `a`, después `f`, después `r`, con el techo bajo (`-`). Marcarlo físicamente.
4. El mismo armado mueve el motor para la prueba del ACS712 (ficha 8).

TIENE QUE DAR
Gira en los dos sentidos, la velocidad sigue al porcentaje y la fuente solo toca el límite en el arranque.

MALA SEÑAL
No gira en un sentido (medio puente muerto). La fuente se va al límite con el motor quieto, o el módulo calienta sin carga (llave en corto).

CUIDADO CON: EL 74HC244 Y EL VCC LÓGICO
El chip de 20 patas junto a la tira de pines es un **buffer 74HC244** (así viene en el IBT-2 típico; confirmalo leyendo la marca en el nuestro). Se alimenta del VCC del módulo, y su umbral alto es ~0,7 de esa tensión:

- **VCC a 5 V:** pide ~3,5 V (3,15 V garantizado recién con 4,5 V de alimentación). El ESP32 garantiza 2,64 V en alto (0,8 × 3,3 V): queda fuera de la hoja.
- **VCC a 3,3 V:** pide ~2,3 V y anda con margen. A su salida, el BTS7960 pide como mucho 2,15 V.

Por eso, **VCC lógico a 3,3 V**. Las entradas del 74HC244 no tienen pull-down: si un cable se suelta, la entrada flota. Por eso van los 10 k a GND.

CUIDADO CON
`0` frena de golpe: es para cuando algo anda mal, pero la verdadera parada de emergencia es apagar la fuente. No inviertas a plena marcha: C1 baja la rampa a cero y deja las dos entradas en bajo antes de cambiar.

Para seguir leyendo en el repo `20-modelo-escala/20-electronica/drivers-bts7960.md` (interfaz, habilitaciones, V1, desacople local) · `30-pcb/mapa-conectores.md` (J10, J11, J15, J19)

8 ACS712: corriente de un motor

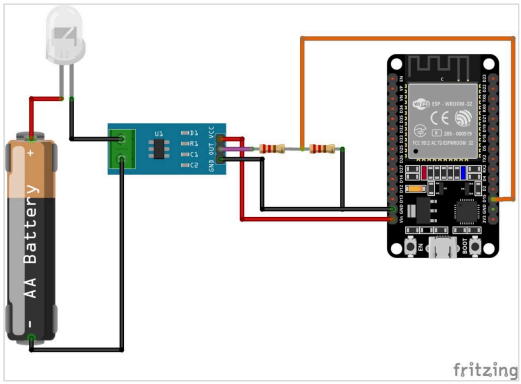
Tarea #7 · Desbloquea: la cadena de medición

El ACS712 convierte corriente en tensión: 66 mV/A alrededor de la mitad de su alimentación. Primero se prueba solo, contra el multímetro, y después con el ADS1115.

QUÉ ES

Sensor de **efecto Hall**: la corriente pasa por un conductor de cobre dentro del chip, genera un campo magnético, y el Hall lo convierte en tensión. Queda aislado del circuito que mide. El **ACS712-30A** da **66 mV/A** con 5 V de alimentación, y el cero está en la **mitad de la alimentación** (2,5 V); por encima es un sentido de giro y por debajo, el otro.

Es proporcional a su alimentación: si el 5 V baja un 2 %, el cero y la sensibilidad bajan un 2 %. Por eso se mide también su 5 V y se corrige. En el rover va en la pata de motor de una rueda delantera, leído por el ADS1115.



Dibujo de Fritzing que circula: ACS712 con divisor y ESP32.

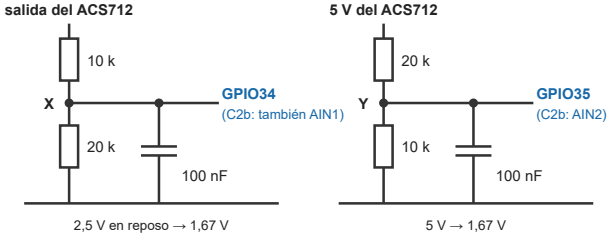
¿HACE FALTA UN DIVISOR? SÍ

Situación	Salida del ACS712 (a 5 V)	¿Pasa 3,3 V?
Reposo	2,5 V	No
Motor trabado, ±5,5 A	2,14 a 2,86 V	No, pero cae en la zona alta y alineal del ADC del ESP32
Más de +12 A, o una falla	hasta ~4,5 V	**Sí**

Con 11 dB, el ADC del ESP32 es lineal entre 0,15 y 2,45 V (Espressif). El **divisor de 2/3** (10 k arriba, 20 k abajo) lleva 0 a 5 V a 0 a 3,33 V y el reposo a ~1,67 V, en la zona buena. Sus 30 k cumplen la carga mínima de 4,7 k del ACS712, y el 10 k separa de su salida el 100 nF que filtra el PWM (el chip admite como mucho 10 nF directo). **Lo que se pierde:** la sensibilidad baja a 44 mV/A, ~20 mA por escalón con el ADC del ESP32 y ~3 mA por cuenta con el ADS1115. El 5 V se lee con un **divisor de 1/3**, las mismas resistencias al revés: 5 V da 1,67 V, en la misma zona.

EN NUESTRA PLACA ES DISTINTO

1. **El divisor está bien**: hace falta.
2. **No uses GPIO15.** Es de ADC2 (que no anda con el Wi-Fi prendido) y además es pin de arranque. Usá **GPIO34** (ADC1).
3. **Esa carga no se ve.** Pila AA y LED dan ~20 mA, o sea ~1,3 mV a la salida: el ACS712-30A no los distingue del ruido. Usá el motor, una lámpara de auto o una resistencia de potencia, de 1 a 5 A.



Los dos divisores de C2a y C2b. En reposo, X e Y quedan casi en la misma tensión.

QUÉ VAMOS A HACER

1. Armado en banco con ESP32 suelto (la única prueba donde se admite el protoboard). La corriente del motor entra y sale por la **bornera** del módulo, con cable de potencia.
2. Multímetro de 10 A **en serie**, como referencia.
3. **C2a** (en el anexo): arrancar sin corriente (tara) y comparar contra el multímetro en varios puntos entre 1 y 5 A.
4. **C2b**: sumar el ADS1115 al mismo armado y comparar los dos caminos en el mismo instante.

TIENE QUE DAR

En reposo, ~1,67 V en GPIO34 y en GPIO35, y una tara cerca de 0,500. La lectura sigue al multímetro en todo el rango, con signo. ADS y ESP32 coinciden, el ADS con menos ruido.

MALA SEÑAL

VCC del sensor fuera de 4,5 a 5,5 V. Lectura que no se mueve con la corriente. Mucho ruido con el PWM (revisar el 100 nF). Una diferencia fija contra el multímetro es esperable: la sensibilidad tiene ±5 % (63 a 69 mV/A, hoja de datos), y por eso se calibra.

OBSERVACIÓN SOBRE LA PLACA (NO SE CAMBIA NADA HOY)

En la placa, el ACS712 va al ADS con 10 k en serie y **sin divisor**. Para la pata de un motor alcanza (2,14 a 2,86 V), y el 10 k protege al ADS, pero por encima de +12 A la lectura se pierde contra los 3,3 V. Queda anotado para revisar en `sensado-corriente.md`.

Para seguir leyendo en el repo `20-modelo-escala/20-electronica/sensado-corriente.md` ("ACS712 de respaldo") · `50-ensayos/instrumentacion.md` (por qué en la pata del motor y no en la batería)

9 Bus I2C y ADS1115

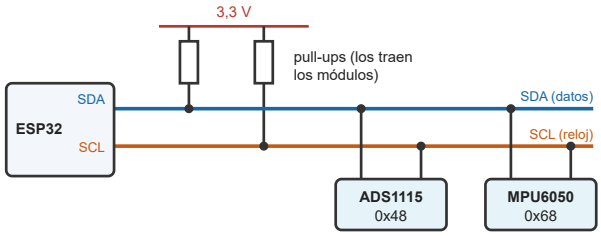
Tarea #7 · Desbloquea: la cadena de medición

I2C son dos cables compartidos por varios chips, cada uno con su dirección. El ADS1115 es un conversor de 16 bits que reemplaza al ADC del ESP32 para medir.

QUÉ ES

I2C: un cable de datos (**SDA**) y uno de reloj (**SCL**). Los chips solo pueden tirar las líneas a masa; las resistencias de **pull-up** las vuelven a subir. En la placa no hay pull-ups: los traen los módulos (GY-521 y ADS1115) y quedan en paralelo. Cada chip tiene una **dirección**: el ADS1115 en **0x48** (ADDR a GND) y el MPU6050 en **0x68**. El **escaneo** llama a cada dirección y anota cuáles contestan.

ADS1115: 16 bits, con amplificador de ganancia programable (**PGA**): es la ganancia que el ADS elige en cada conversión, y se ajusta al rango de cada señal. El diseño usa **±2,048 V en el TMCS** (AIN0: 0,63 mA por cuenta) y **±4,096 V en el ACS712** (AIN1 y AIN2: con ±2,048 V sus 2,5 V saturarían). Las dos le ganan a los 8 mA por escalón del ADC del ESP32, que además es alineal. Además, cada conversión integra un tiempo y promedia el PWM.



El bus I2C de la placa. Qué GPIO son SDA y SCL: en `mapa-gpio.md`.

QUÉ VAMOS A HACER

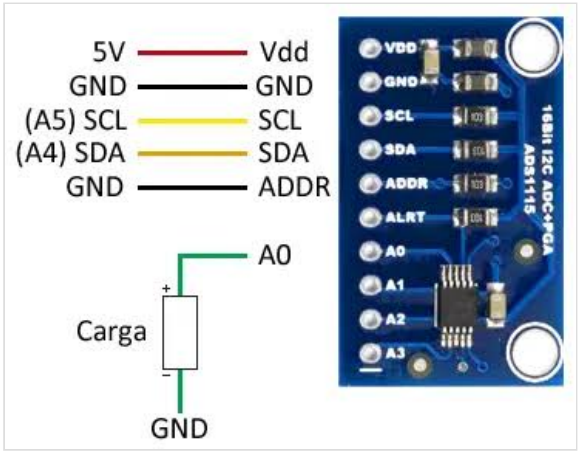
En la placa, con `test-ads1115.ino`: primero el escaneo (tienen que aparecer 0x48 y 0x68), después los cuatro canales cada 0,5 s. El MPU6050 tiene su propio `test-mpu6050.ino`.

Canal	Señal	En reposo
AIN0	TMCS1126 (ficha 10)	~1,65 V
AIN1	ACS712	~2,5 V
AIN2	5 V del ACS712 / 2	~2,5 V
AIN3	libre	cualquiera (flota)

MALA SEÑAL

Escaneo vacío: revisar 3,3 V, GND, SDA y SCL. Aparece uno solo: revisar el que falta. AIN2 lejos de 2,5 V: revisar el regulador del ACS712. AIN0 lejos de 1,65 V: revisar el cable desde C8.

`test-ads1115.ino` usa ±4,096 V en los cuatro canales solo por simplicidad: en el TMCS da 1,25 mA por cuenta en vez de 0,63. La asignación por canal está en `instrumentacion.md`.



Módulo ADS1115 conectado a un Arduino. Imagen de internet.

EN NUESTRA PLACA ES DISTINTO

1. **Vdd a 3,3 V, no a 5 V.** Con 5 V, los pull-ups del módulo suben SDA y SCL a 5 V, sobre las entradas del ESP32.
2. **SDA y SCL no son A4 y A5** (eso es de Arduino): son los GPIO de `mapa-gpio.md`.
3. ADDR a GND → 0x48: coincide con lo nuestro.

Para seguir leyendo en el repo `20-modelo-escala/40-firmware/test-ads1115/test-ads1115.ino` (comentado línea por línea) · `50-ensayos/instrumentacion.md` (canales del ADS1115) · `40-firmware/mapa-gpio.md`

10 TMCS1126: código de lectura

Tarea #8 · Cierra: V9, V11, calibración

El TMCS1126 mide la corriente de tracción: 100 mV/A alrededor de 1,65 V, y al consumir la salida baja. Se tara en cada sesión.

QUÉ ES

Otro sensor Hall, U3, montado en la placa. Mide **solo la rama de tracción** (los cuatro drivers), no el total. Variante **B4**: **100 mV/A**, cero en **1,65 V**, rango de ±15,5 A. Por cómo está cableado, **el signo queda invertido**: al descargar la batería, la salida **baja**. Con PWM mide corriente de batería, que es aproximadamente D · I_motor: para la del motor hay que dividir por el ciclo de trabajo D.

`I = (V_reposo − V) / 0,1`. **V_reposo se mide, no se supone**: se toma con el bus energizado y los motores parados, e incluye el consumo lógico de los drivers (10 a 30 mA).

TIENE QUE DAR

Reposo cerca de 1,65 V. Con una carga conocida, la tensión baja y la corriente da positiva y coincide con el multímetro.

MALA SEÑAL

La tensión sube con la carga: el signo es el opuesto al previsto (V11; se corrige en el código). VOC lejos de 3,0 V (V9).

QUÉ VAMOS A HACER

1. Agregar estas líneas sobre `leerCanal()` de `test-ads1115.ino`:

BORRADOR SIN PROBAR

```
// tara: bus energizado, motores parados
float vReposo = 0, v;
for (int k = 0; k < 32; k++) { leerCanal(0, v); vReposo += v; }
vReposo /= 32;
leerCanal(0, v);
float iTraccion = (vReposo - v) / 0.1f;  // A; + = descarga
```

2. Carga conocida (lámpara de auto o resistencia de potencia) contra el multímetro en serie: confirmar el signo (V11).
3. Medir VOC con el multímetro: el cálculo da 3,0 V (V9).

CUIDADO CON

Tarar al inicio de cada sesión. Satura a ~15,5 A (V8): con cuatro motores trabados no entra en rango. Al encender tarda 34 ms en dar una salida válida.

Para seguir leyendo en el repo `20-modelo-escala/20-electronica/sensado-corriente.md` (dónde está, signo, V8, V9, calibración) · `tmcs1126-datasheet.pdf` · `50-ensayos/instrumentacion.md` (corrección por D)

11 Encoders con PCNT

El PCNT ya está explicado en su guía. Acá solo hay tres números para llevar en la cabeza y qué leer de esa guía antes de ir.

LO BÁSICO

Encoder en cuadratura: dos ondas cuadradas, A y B, corridas un cuarto de período (90°). La frecuencia da la velocidad; **cuál cambia primero da el sentido**. **"El ESP32 cuenta solo":** el PCNT es un contador de hardware que suma o resta en cada flanco sin interrumpir al procesador; el programa lee el número cuando quiere.

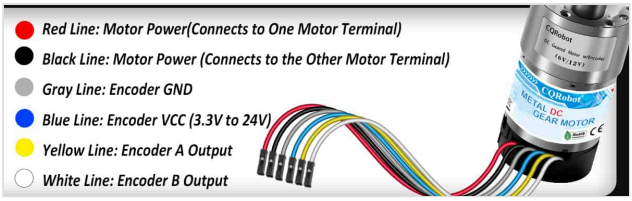
Número	De dónde sale
5760 cuentas por vuelta de rueda	16 pulsos por vuelta del motor × 4 flancos × 90 de reducción
Desborde cada **2,8 s**	Contador de 16 bits (±32767) a 11520 cuentas/s (120 rpm). Hay que acumular en una variable más grande
Filtro de glitch	Descarta pulsos más cortos que un umbral (1,25 µs en la guía), frente a los 86 µs del evento real más corto. Es gratis

GUÍA DE LECTURA DE `PCNT_ESP32_Guia_Rover.pdf`

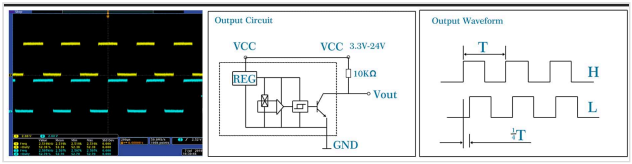
1. **§1 y §2** antes de ir: la idea en una página y la cuadratura.
2. **§7, primer recuadro:** la versión del core. Los ejemplos piden la **3.x**.
3. **Ej1** (PCNT mínimo): para entender los cinco pasos.
4. **Ej4** (diagnóstico de los 8 cables): **el del labo**. Cada cable da 2880 flancos por vuelta de rueda.
5. **Checklist §12**, pasos 2 a 5: cables, signo de cada rueda, velocidad a pleno y ruido.

"CONTAR MAL"

Cuentas falsas con el motor andando y PWM, o una posición que deriva con la rueda quieta. El orden de corrección: filtro de glitch → pull-ups de 2,2 k → RC de 2,2 k + 1 nF (nunca 10 nF).



Colores: rojo y negro, motor; **gris, GND del encoder; azul, VCC** (3,3 a 24 V; en la placa, 3,3 V); **amarillo, A; blanco, B**. Imagen del fabricante (CQRobot).



Captura real de A y B (amarillo y celeste), el circuito de salida y la forma de onda. Imagen del fabricante.

EN NUESTRA PLACA ES DISTINTO

- Los cables de fábrica traen **Dupont**: en el rover van con conector con traba.
- La captura es con el encoder a 5 V; en la placa va a 3,3 V.
- **Dato de la hoja:** salida de colector abierto con **pull-up interno de 10 k**, y **10 mA como máximo por encoder**. Con el 4,7 k de la placa quedan ~3,2 k en paralelo.

CUIDADO CON: LOS PINES DE EJ4

Los pines de `PINES[]` son de ejemplo y **no son los de la placa**: en la placa, 32, 33, 25 y 26 son la batería, la corriente y el PWM del driver 1. Antes de cargarlo, reemplazalos por los 8 de los encoders, según `mapa-gpio.md`. Ej4 usa las 8 unidades PCNT: no lo mezcles con otro código que use PCNT.

Para seguir leyendo en el repo `20-modelo-escala/20-electronica/PCNT_ESP32_Guia_Rover.pdf` · `encoders-pcnt.md` (acondicionamiento y pendientes) · `30-pcb/mapa-conectores.md` (J8, J9, J16, J18) · `40-firmware/mapa-gpio.md`

Anexo · Código C1 — prueba de un módulo BTS7960 en banco SIN PROBAR

Core **3.x** (`ledcAttach`), 20 kHz y 10 bits. Pines del ESP32 suelto, no los de `mapa-gpio.md`. Compila sin errores ni advertencias; no se probó con hardware.

```
/*
  C1_BTS7960.ino - prueba de un modulo BTS7960 (IBT-2) en banco
  estado: SIN PROBAR. Borrador para el labo, 26-oct-2026.
  Core Arduino-ESP32 3.x. ESP32 suelto, NO la placa del rover.

  Conexion (pines del ESP32 suelto, no los de mapa-gpio.md):
    GPIO25 -> RPWM          GPIO26 -> LPWM
    3V3 -> VCC, R_EN, L_EN  (VCC logico a 3,3 V, como en la placa)
    GND -> GND              R_IS y L_IS sin conectar
    10 k de RPWM a GND y de LPWM a GND, del lado del modulo
  Potencia: fuente LIMITADA EN CORRIENTE a B+ y B-;
            motor a M+ y M-.

  Comandos por el monitor serie (115200):
    a  rampa adelante (PWM en RPWM, LPWM en bajo)
    r  rampa atras    (PWM en LPWM, RPWM en bajo)
    f  rampa a cero y frena (las dos entradas en bajo)
    0  corte inmediato: las dos en bajo YA (frena de golpe)
    s  secuencia: adelante, frenado, atras, frenado
    + -  techo de la rampa de a 10 % (arranca en 50 %)

  Antes de cambiar de sentido baja la rampa a cero y deja las
  dos entradas en bajo PAUSA_MS. Con EN fijo en alto, "las dos
  en bajo" es frenado contra masa, no rueda libre.
*/

#include <Arduino.h>
#include "esp_arduino_version.h"

#if ESP_ARDUINO_VERSION_MAJOR < 3
#error "Usa ledcAttach(): requiere el core Arduino-ESP32 3.x"
#endif

constexpr int PIN_RPWM = 25;
constexpr int PIN_LPWM = 26;

constexpr uint32_t FREC_PWM = 20000;  // 20 kHz: no se escucha
constexpr uint8_t  BITS_PWM = 10;     // 0..1023
constexpr int CUENTA_MAX = (1 << BITS_PWM) - 1;

constexpr uint32_t PASO_MS = 10;   // la rampa avanza cada 10 ms
constexpr int PASO_CUENTAS = 10;   // de 0 a 100 % en ~1 s
constexpr uint32_t PAUSA_MS = 300; // las dos en bajo al invertir

enum Sentido { FRENADO, ADELANTE, ATRAS };
const char *NOMBRE[] = {"frenado", "adelante", "atras"};

Sentido sentidoActual = FRENADO;  // que entrada lleva el PWM
Sentido sentidoPedido = FRENADO;  // a donde hay que ir
int duty = 0;                     // ciclo aplicado, en cuentas
int dutyObjetivo = 0;
int porcentajeMax = 50;           // techo de la rampa
bool enPausa = false;
uint32_t inicioPausa = 0;

// Secuencia de 's'. Cada paso dura 'ms', rampa incluida.
struct Paso { Sentido sentido; uint32_t ms; };
const Paso SECUENCIA[] = {
  {ADELANTE, 4000}, {FRENADO, 2000},
  {ATRAS,    4000}, {FRENADO,    0},
};
constexpr int N_PASOS = sizeof(SECUENCIA) / sizeof(Paso);
bool enSecuencia = false;
int pasoSec = 0;
uint32_t inicioPaso = 0;

void aplicar() {
  ledcWrite(PIN_RPWM, sentidoActual == ADELANTE ? duty : 0);
  ledcWrite(PIN_LPWM, sentidoActual == ATRAS    ? duty : 0);
}

void pedir(Sentido s) {
  sentidoPedido = s;
  dutyObjetivo = (s == FRENADO) ? 0
               : CUENTA_MAX * porcentajeMax / 100;
}

// Un paso de rampa. Si cambia el sentido: bajar a cero,
// pausa con las dos entradas en bajo, recien ahi cambiar.
void pasoRampa() {
  if (sentidoPedido == sentidoActual) {
    enPausa = false;
  } else {
    if (duty > 0) {
      duty = max(0, duty - PASO_CUENTAS);
      aplicar();
      return;
    }
    if (!enPausa) {
      sentidoActual = FRENADO;
      aplicar();
      enPausa = true;
      inicioPausa = millis();
      return;
    }
    if (millis() - inicioPausa < PAUSA_MS) return;
    enPausa = false;
    sentidoActual = sentidoPedido;
  }
  if (duty < dutyObjetivo)
    duty = min(dutyObjetivo, duty + PASO_CUENTAS);
  else if (duty > dutyObjetivo)
    duty = max(dutyObjetivo, duty - PASO_CUENTAS);
  aplicar();
}

void avanzarSecuencia() {
  if (!enSecuencia) return;
  if (millis() - inicioPaso < SECUENCIA[pasoSec].ms) return;
  pasoSec++;
  if (pasoSec >= N_PASOS) {
    enSecuencia = false;
    Serial.println("# Secuencia terminada");
    return;
  }
  pedir(SECUENCIA[pasoSec].sentido);
  inicioPaso = millis();
}

void ayuda() {
  Serial.println("# a adelante | r atras | f frenar | 0 corte");
  Serial.println("# s secuencia | + / - techo de la rampa");
}

void atenderSerie() {
  if (!Serial.available()) return;
  char c = Serial.read();
  if (c == '\n' || c == '\r') return;
  // cualquier comando manual corta la secuencia
  if (c != '+' && c != '-') enSecuencia = false;

  switch (c) {
    case 'a': pedir(ADELANTE); break;
    case 'r': pedir(ATRAS);    break;
    case 'f': pedir(FRENADO);  break;
    case '0':
      sentidoPedido = sentidoActual = FRENADO;
      duty = dutyObjetivo = 0;
      enPausa = false;
      aplicar();
      break;
    case 's':
      pasoSec = 0;
      pedir(SECUENCIA[0].sentido);
      inicioPaso = millis();
      enSecuencia = true;
      break;
    case '+':
    case '-':
      porcentajeMax += (c == '+') ? 10 : -10;
      porcentajeMax = constrain(porcentajeMax, 10, 100);
      if (sentidoPedido != FRENADO) pedir(sentidoPedido);
      Serial.printf("# Techo: %d %%\n", porcentajeMax);
      break;
    default:
      ayuda();
  }
}

void setup() {
  Serial.begin(115200);
  delay(300);
  Serial.println("\n# == C1: modulo BTS7960 (sin probar) ==");
  bool ok = ledcAttach(PIN_RPWM, FREC_PWM, BITS_PWM);
  ok = ok && ledcAttach(PIN_LPWM, FREC_PWM, BITS_PWM);
  if (!ok) {
    Serial.println("# ERROR: ledcAttach fallo (core? pines?)");
    while (true) delay(1000);
  }
  aplicar();  // arranca con las dos entradas en bajo
  ayuda();
}

void loop() {
  atenderSerie();

  static uint32_t ultimoPaso = 0;
  if (millis() - ultimoPaso >= PASO_MS) {
    ultimoPaso = millis();
    avanzarSecuencia();
    pasoRampa();
  }

  // Estado: solo cuando cambia, como mucho cada 200 ms
  static uint32_t ultimoInforme = 0;
  static int dutyInf = -1;
  static Sentido sentidoInf = FRENADO;
  bool cambio = duty != dutyInf || sentidoActual != sentidoInf;
  if (cambio && millis() - ultimoInforme >= 200) {
    ultimoInforme = millis();
    dutyInf = duty;
    sentidoInf = sentidoActual;
    Serial.printf("%-9s %3d %%\n", NOMBRE[sentidoActual],
                  duty * 100 / CUENTA_MAX);
  }
}
```

Anexo · Código C2a — ACS712 con el ADC del ESP32 SIN PROBAR

Usa `analogReadMilliVolts()`, que aplica la calibración de fábrica del chip. Compila sin errores ni advertencias (core 2.0.17; las funciones son las mismas en la 3.x).

```
/*
  C2a_ACS712_ADC.ino - ACS712-30A con el ADC del ESP32
  estado: SIN PROBAR. Borrador para el labo, 26-oct-2026.
  ESP32 suelto en protoboard, NO la placa del rover.

  Conexion (el mismo armado sirve para C2b):
    ACS712 VCC <- 5V del devkit (pin VIN, viene del USB)
    ACS712 GND -> GND
    ACS OUT -> 10 k -> nodo X -> 20 k -> GND
               nodo X -> GPIO34, con 100 nF a GND
    5V      -> 20 k -> nodo Y -> 10 k -> GND
               nodo Y -> GPIO35, con 100 nF a GND
  Divisor de la salida 2/3: reposo 2,5 V -> 1,67 V.
  Divisor del 5 V 1/3:      5 V -> 1,67 V.
  En reposo GPIO34 y GPIO35 leen casi lo mismo.

  La salida del ACS712 es proporcional a su alimentacion
  (cero en VCC/2, 66 mV/A con VCC = 5 V). El 5 V del USB no
  es exacto: se mide y se corrige.

  Uso: arrancar SIN corriente (tara). 't' vuelve a tarar.
  Formato etiqueta:valor, sirve en el Serial Plotter.
*/

#include <Arduino.h>

constexpr int PIN_SALIDA = 34;  // ADC1: anda con Wi-Fi
constexpr int PIN_VCC    = 35;  // ADC1

constexpr float K_SALIDA = 1.5f;    // inversa del divisor 2/3
constexpr float K_VCC    = 3.0f;    // inversa del divisor 1/3
constexpr float SENS_5V  = 0.066f;  // V/A con VCC = 5,0 V
constexpr int N_MUESTRAS = 64;      // promedio por lectura

float ceroRel = 0.5f;  // salida/VCC sin corriente (tara)

// Promedio en mV. analogReadMilliVolts usa la calibracion
// de fabrica del chip.
float leerMilivoltios(int pin) {
  uint32_t suma = 0;
  for (int i = 0; i < N_MUESTRAS; i++)
    suma += analogReadMilliVolts(pin);
  return (float)suma / N_MUESTRAS;
}

void leerSensor(float &vSalida, float &vcc) {
  vSalida = leerMilivoltios(PIN_SALIDA) * K_SALIDA / 1000;
  vcc     = leerMilivoltios(PIN_VCC)    * K_VCC    / 1000;
}

void tarar() {
  float suma = 0, vSalida, vcc;
  for (int i = 0; i < 20; i++) {
    leerSensor(vSalida, vcc);
    suma += vSalida / vcc;
    delay(10);
  }
  ceroRel = suma / 20;
  Serial.printf("# Tara: cero en %.4f de VCC (ideal 0,5)\n",
                ceroRel);
}

void setup() {
  Serial.begin(115200);
  delay(300);
  Serial.println("\n# == C2a: ACS712 con ADC (sin probar) ==");
  analogReadResolution(12);
  // 11 dB: zona lineal hasta ~2,45 V
  analogSetPinAttenuation(PIN_SALIDA, ADC_11db);
  analogSetPinAttenuation(PIN_VCC, ADC_11db);
  Serial.println("# Tarando: sin corriente por el sensor...");
  tarar();
}

void loop() {
  if (Serial.available() && Serial.read() == 't') tarar();

  float vSalida, vcc;
  leerSensor(vSalida, vcc);
  float sens = SENS_5V * vcc / 5.0f;  // escala con VCC
  float corriente = (vSalida - ceroRel * vcc) / sens;

  Serial.printf("Vout:%.3f,VCC:%.3f,I:%.3f\n",
                vSalida, vcc, corriente);
  if (vcc < 4.5f || vcc > 5.5f)
    Serial.println("# OJO: VCC del ACS fuera de 4,5..5,5 V");
  if (vSalida / K_SALIDA > 2.45f)
    Serial.println("# OJO: GPIO34 > 2,45 V, ADC alineal");
  delay(250);
}
```

Anexo · Código C2b — ACS712 con el ADS1115, comparado con el ADC del ESP32 SIN PROBAR

Reutiliza `leerCanal()` de `test-ads1115.ino`, sin librerías. Mismo armado que C2a, más el ADS1115.

```
/*
  C2b_ACS712_ADS1115.ino - ACS712-30A con el ADS1115,
  comparado en el mismo instante con el ADC del ESP32
  estado: SIN PROBAR. Borrador para el labo, 26-oct-2026.
  ESP32 suelto en protoboard, NO la placa del rover.

  Armado de C2a, mas el ADS1115:
    VDD <- 3V3 (NUNCA 5 V)   GND -> GND
    ADDR -> GND (0x48)       SDA -> GPIO21   SCL -> GPIO22
    nodo X (salida del ACS por 2/3) -> AIN1 (y a GPIO34)
    nodo Y (5 V del ACS por 1/3)    -> AIN2 (y a GPIO35)
  AIN1 y AIN2, como en la placa. Con los divisores ninguna
  entrada pasa de 3,33 V.

  configCanal() y leerCanal() son copia de
  20-modelo-escala/40-firmware/test-ads1115/test-ads1115.ino
*/

#include <Arduino.h>
#include <Wire.h>

constexpr int PIN_SDA = 21;
constexpr int PIN_SCL = 22;
constexpr uint8_t ADS_ADDR = 0x48;
constexpr uint8_t REG_CONVERSION = 0x00;
constexpr uint8_t REG_CONFIG     = 0x01;
constexpr float VOLTIOS_POR_CUENTA = 4.096f / 32768.0f;

constexpr int CANAL_SALIDA = 1, CANAL_VCC = 2;  // ADS
constexpr int PIN_SALIDA = 34, PIN_VCC = 35;    // ESP32

constexpr float K_SALIDA = 1.5f;    // inversa del divisor 2/3
constexpr float K_VCC    = 3.0f;    // inversa del divisor 1/3
constexpr float SENS_5V  = 0.066f;  // V/A con VCC = 5,0 V
constexpr int N_ADC = 64;  // promedio del ADC del ESP32
constexpr int N_ADS = 4;   // promedio del ADS (~9 ms c/u)

float ceroAds = 0.5f, ceroEsp = 0.5f;  // salida/VCC (tara)

// ---- copia de test-ads1115.ino (PGA +-4,096 V) ----
uint16_t configCanal(uint8_t canal) {
  return 0x8000 | ((0x4 + canal) << 12) | 0x0200 | 0x0100
       | 0x0080 | 0x0003;
}

bool leerCanal(uint8_t canal, float &voltios) {
  uint16_t cfg = configCanal(canal);
  Wire.beginTransmission(ADS_ADDR);
  Wire.write(REG_CONFIG);
  Wire.write(cfg >> 8);
  Wire.write(cfg & 0xFF);
  if (Wire.endTransmission() != 0) return false;
  delay(9);
  Wire.beginTransmission(ADS_ADDR);
  Wire.write(REG_CONVERSION);
  if (Wire.endTransmission() != 0) return false;
  if (Wire.requestFrom(ADS_ADDR, (uint8_t)2) != 2)
    return false;
  uint8_t alto = Wire.read();
  uint8_t bajo = Wire.read();
  int16_t cuentas = (int16_t)((alto << 8) | bajo);
  voltios = cuentas * VOLTIOS_POR_CUENTA;
  return true;
}
// ---- fin de la copia ----

bool leerAds(float &vSalida, float &vcc) {
  float a = 0, b = 0, v;
  for (int i = 0; i < N_ADS; i++) {
    if (!leerCanal(CANAL_SALIDA, v)) return false;
    a += v;
    if (!leerCanal(CANAL_VCC, v)) return false;
    b += v;
  }
  vSalida = a / N_ADS * K_SALIDA;
  vcc     = b / N_ADS * K_VCC;
  return true;
}

void leerEsp(float &vSalida, float &vcc) {
  uint32_t a = 0, b = 0;
  for (int i = 0; i < N_ADC; i++) {
    a += analogReadMilliVolts(PIN_SALIDA);
    b += analogReadMilliVolts(PIN_VCC);
  }
  vSalida = (float)a / N_ADC * K_SALIDA / 1000;
  vcc     = (float)b / N_ADC * K_VCC / 1000;
}
```

```
float corriente(float vSalida, float vcc, float cero) {
  return (vSalida - cero * vcc) / (SENS_5V * vcc / 5.0f);
}

void tarar() {
  float sa = 0, se = 0, vs, vc;
  for (int i = 0; i < 10; i++) {
    if (!leerAds(vs, vc)) {
      Serial.println("# Tara: ADS1115 sin respuesta");
      return;
    }
    sa += vs / vc;
    leerEsp(vs, vc);
    se += vs / vc;
  }
  ceroAds = sa / 10;
  ceroEsp = se / 10;
  Serial.printf("# Tara: ADS %.4f, ESP32 %.4f (ideal 0,5)\n",
                ceroAds, ceroEsp);
}

void setup() {
  Serial.begin(115200);
  delay(300);
  Serial.println("\n# == C2b: ACS712 con ADS1115 (sin probar) ==");
  analogReadResolution(12);
  analogSetPinAttenuation(PIN_SALIDA, ADC_11db);
  analogSetPinAttenuation(PIN_VCC, ADC_11db);

  Wire.begin(PIN_SDA, PIN_SCL, 100000);
  Wire.beginTransmission(ADS_ADDR);
  if (Wire.endTransmission() != 0)
    Serial.println("# Sin ADS en 0x48: 3,3 V, GND, SDA, SCL?");

  Serial.println("# Tarando: sin corriente por el sensor...");
  tarar();
}

void loop() {
  if (Serial.available() && Serial.read() == 't') tarar();

  float vsA, vcA, vsE, vcE;
  if (!leerAds(vsA, vcA)) {
    Serial.println("# ADS1115 sin respuesta en 0x48");
    delay(1000);
    return;
  }
  leerEsp(vsE, vcE);

  float iAds = corriente(vsA, vcA, ceroAds);
  float iEsp = corriente(vsE, vcE, ceroEsp);
  Serial.printf("I_ADS:%.3f,I_ESP:%.3f,VCC_ADS:%.3f,"
                "VCC_ESP:%.3f\n", iAds, iEsp, vcA, vcE);
  delay(250);
}
```